Trail Sample Exam Questions

I. Multiple Choice – For each of the following questions, please select the best answer from the given choices. Remember there is only **ONE** best answer to each question.

1. If there are n ($n \geq 2$) processes in the system, and each process needs to use 2 critical resources of a certain type, then the total number of such resources required for the system to avoid deadlock is at least **C**
   a) n^2
   b) n
   c) $n + 1$
   d) $2n$
2. Among the following statements about deadlock, the correct one is: **B**
   I. Deadlock can be resolved by preempting process resources
   II. Deadlock prevention methods can ensure that the system does not occur deadlock
   III. Banker's Algorithm can judge whether the system is in deadlock state
   IV. When a deadlock occurs in the system, there must be two or more processes in the blocked state
   a) Only II, III
   b) Only I, II, IV
   c) Only I, II, III
   d) I, III, IV only

3. What is the reason for having multi-level page tables? **B**
   a) To reduce the possibilities of internal and external fragmentations
   b) To avoid storing the entire Page Table as a contiguous array
   c) Make virtual memory possible
   d) To increase flexibility in allocating main memory

4. In the following description about threads, the wrong one is **B**
   a) The scheduling of kernel-level threads is done by the operating system
   b) The operating system creates a thread control block for each user-level thread
   c) Switching between user-level threads is more efficient than switching between kernel-level threads
   d) User-level threads can be implemented on operating systems that do not support kernel-level threads

5. Which of the following statement is False about deadlock detection policies? **D**

I. Wait-die scheme is a non-preemptive technique while wound-wait scheme is a preemptive technique for deadlock prevention.
II. If Ti requests a lock held by Tj and Ti is older than Tj then in wait-die scheme Ti will wait and in wound-wait scheme Tj will wait.
III. There is deadlock in the system if and only if there exists a cycle in the wait-for-graph.

a) I
b) II
c) I and II
d) II and III

6. Which of the following statements is correct with respect to paging? **B**
   a) Paging incurs no memory overheads
   b) Paging helps solve the issue of external fragmentation
   c) Paging size has no impact on internal fragmentation
   d) Multi-level paging is necessary to support pages of different sizes

7. Which of the following best describes segmentation in memory management? **B**
   a) Dividing memory into fixed-size blocks and mapping them to pages.
   b) Dividing memory into variable-sized segments based on the type of data or code.
   c) The process of allocating memory dynamically to processes.
   d) A technique where memory is shared between multiple processes without isolation.

8. Which of the following statements about the parent process and the child process is wrong? **B**
   a) The parent process and the child process can execute concurrently.
   b) The parent process and the child process share the virtual address space.
   c) The parent process and the child process have different process control blocks.
   d) The parent process and the child process cannot use the same critical resource at the same time.

9. The number of tracks of the disk system is 200 (0~199), and the magnetic head is currently on track 184. The track numbers corresponding to the disk access requests made by the user process are 184, 187, 176, 182, and 199 in sequence. If the Shortest Seek Time First (SSTF) is used to schedule disk access, the distance (number of tracks) that the head moves is **C**
   a) 37
   b) 38
   c) 41

d) 42

10. '*m*' processes share '*n*' resources of the same type. The maximum need of each process doesn't exceed '*n*' and the sum of all their maximum needs is always less than *m* + *n*. In this setup, deadlock _____**A**_____
    a) can never occur
    b) may occur
    c) has to occur
    d) none of the mentioned

II. Short Answer Questions

1. Please compare the two deadlock prevention methods: wait-die and wound-wait.

   Solution:

   Wait–die: Older requester waits; younger requester dies (aborts, keeps its timestamp). Non-preemptive—lock holders aren't rolled back—so it's simpler but causes more young-transaction aborts under contention.

   Wound–wait: Older requester wounds (preempts/aborts) the younger holder; younger requester waits. Preemptive—fewer aborts overall and better throughput, but added complexity to safely roll back holders.

   Both are deadlock-free via timestamps: wait–die is conservative; wound–wait is aggressive.

2. Please explain why operating systems introduce virtual memory and how virtual memory works.

Solution:

Why virtual memory? Per-process isolation and protection with a simple, large, contiguous address space. Run programs larger than RAM via paging to disk → better utilization/multiprogramming. Efficient sharing (libraries), copy-on-write, memory-mapped files, and ASLR.

How it works?

CPU issues virtual addresses → MMU + TLB translate using a per-process page table.

Memory is split into pages (e.g., 4 KiB) mapped to RAM frames or backing store (disk).

On missing page → page fault → OS loads it; if RAM full, evict a page (CLOCK/LRU), write back if dirty.

PTE bits (valid, R/W/X, user/supervisor) enforce protection; shared mappings and copy-on-write handle sharing efficiently.

Overload can cause thrashing (excessive paging).

3. Please explain the differences between children processes and threads.

Solutions:

A child process has its own virtual address space and kernel resources; a thread shares its process's address space and most resources. Because of this, processes provide strong isolation (memory faults don't cross boundaries), while threads can corrupt each other if synchronization is wrong.

Creation, teardown, and context switches are heavier for processes and lighter for threads. Inter-process communication needs IPC mechanisms (pipes, sockets, shared memory), whereas threads communicate by directly reading/writing shared variables, guarded by locks/atomics. Both can run in parallel on multiple cores, but threads are the usual choice for fine-grained in-process parallelism; processes are preferred for reliability, security, and fault containment (e.g., microservices, privilege separation).

III. Please indicate whether the below statement is **true or false:**

1) In a system using a page table, a page fault occurs when the system cannot find the physical address corresponding to a virtual address in the page table. ____ **true** _____
2) In a system with both paging and segmentation, paging handles external fragmentation and segmentation handles internal fragmentation. ____ **true** ____
3) In a multi-level feedback queue scheduling algorithm, a process that uses more CPU time is moved to a lower-priority queue, while a process that uses less CPU time is moved to a higher-priority queue. _____ **true** _____
4) A semaphore is a type of synchronization mechanism that can be used to solve the producer-consumer problem by ensuring mutual exclusion. _____ **true** _____
5) The "Ready" state of a process in an operating system means that the process is currently executing instructions. _____ **false** _____

IV. A bank provides **1** service window and **10** seats for customers to wait. When the customer arrives at the bank, if there is an empty seat, he will go to the number machine to get a number and wait for the number to be called. **Only one customer is allowed to use the machine at a time.** When the bank staff is free, he selects a

customer by calling the number and serve him. The activity process of customers and the bank staff is described as follows

```
Cobegin {
   Processs of Customer i {
      Get a number from the number machine;
      Wait for the number to be called;
      Served by the bank staff;
   }

   Process of the BankStaff {
      While (TRUE) {
         Select a customer by calling the number;
         Serve the selected customer;
      }
   }
} coend
```

Please define needed semaphores and variables and use the `wait(), signal()` operations to realize the mutual exclusion and synchronization in the above process, and explain the meaning of the used semaphores and initial value.

Solutions:

Semaphores & variables

`seatsMutex` = 1 — binary; protects freeSeats.

`machine` = 1 — binary; mutual exclusion for the number machine.

`customers` = 0 — counting; number of waiting customers (in seats).

`call` = 0 — counting; staff "calls a number" to wake exactly one waiting customer.

`int freeSeats` = 10 — available waiting seats.

```
process Customer_i() {
  wait(seatsMutex);
  if (freeSeats > 0) {
     freeSeats--;                // take a seat
     signal(customers);          // announce presence
     signal(seatsMutex);

     wait(machine);      // use number machine exclusively
     get_number();
     signal(machine);

     wait(call);         // wait until staff calls your number
       // go to the service window and get served
       be_served();
  } else {
```

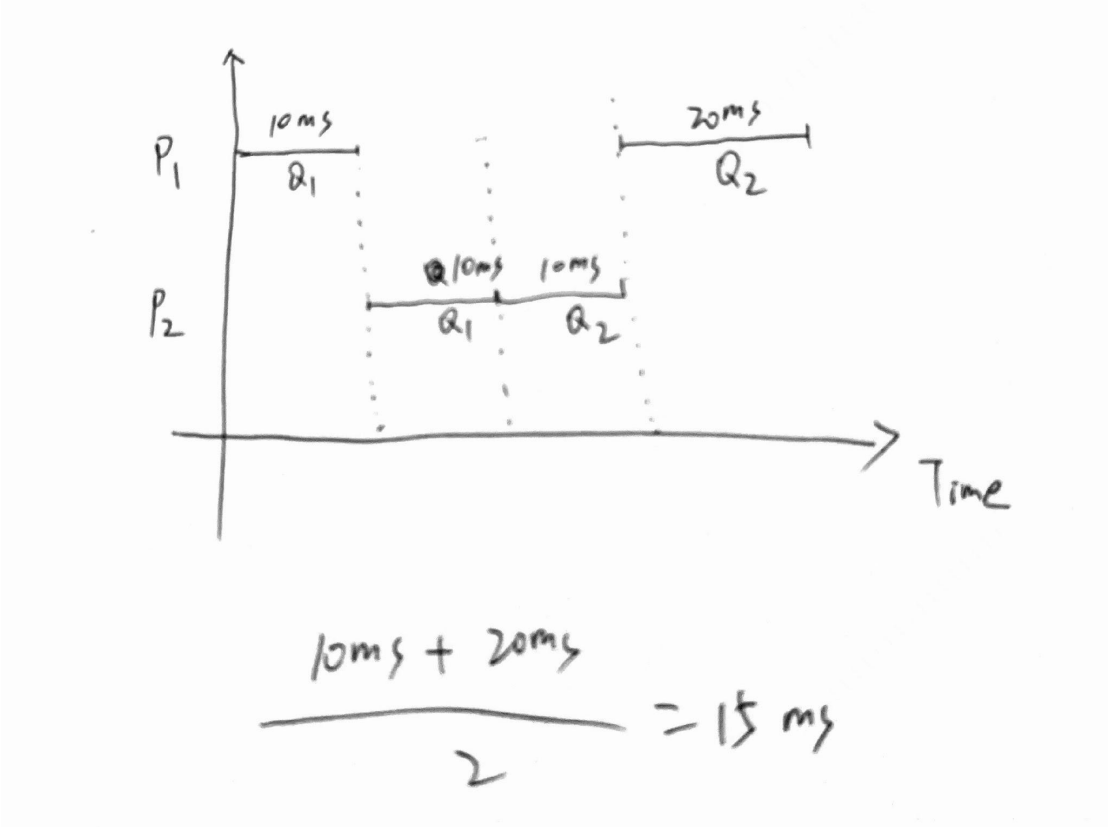
```
     signal(seatsMutex);   // no seat → leave (or retry later)
     leave();
   }
}

process BankStaff() {
   while (TRUE) {
      wait(customers);       // block if no waiting customers
      wait(seatsMutex);
      freeSeats++;  // one seat frees up as a customer approaches
      signal(call);               // call exactly one customer
      signal(seatsMutex);

      serve_selected_customer();   // serve at the single window
   }
}
```

V. The system uses a two-level feedback queue scheduling algorithm for process scheduling. The ready queue Q1 employs a round-robin scheduling algorithm with a time slice of 10ms; the ready queue Q2 uses a shortest job first scheduling algorithm; the system prioritizes scheduling processes in queue Q1, and only schedules processes in Q2 when Q1 is empty; newly created processes first enter Q1; if a process in Q1 executes for one time slice and has not finished, it is then transferred to Q2. If both Q1 and Q2 are currently empty, the system will sequentially create processes P1 and P2 before starting process scheduling. P1 and P2 require 30ms and 20ms of CPU time, respectively. Hence, what is the average waiting time for processes P1 and P2 in the system?

Solutions:



VI. Use Banker's Algorithm to check whether a resources request is safe (to be granted) or not safe (to be rejected) in a specific situation ...

VII: Completion by Matching. Identify the letter of the choice that best complete each statement.
(Just for Question Tyoe ONLY!)

A. Availability
B. Activity
C. Client-server
D. Context
E. Development
F. Ethnography
G. Equivalence partition
H. Functional requirements
I. Heterogeneity
J. Interaction
K. Open-source
L. Pair programming
M. Pipe and filter
N. Reuse
O. Scenario testing
P. Sociotechnical systems
Q. Specification
R. System requirements
S. Technical systems
T. Test-driven
U. Use-case
V. User interfaces
W. User requirements
X. Validation
Y. Verification
Z. Workflow

_O__ 5. _____ is an approach to release testing where you write a story describing how a system may be used and design tests based on the sequence of events in the scenario.
_P__ 6. _____ is/are self-aware and include defined operational processes and procedures.
_A__ 7. _____ is the ability of a system to deliver services when requested.
_L__ 19. Requirements expressed as scenarios, _____, and test-first development are three important characteristics of extreme programming.
_W__ 20. _____ are statements in a language that is understandable to a user of what services the system should provide and the constraints under which it operates.